

BÁO CÁO ĐỒ ÁN / TIỂU LUẬN

DỰ ÁN HEALTHCARE MICROSERVICES

Phân tích use-case, kiến trúc thiết kế, hiện thực mã nguồn và kết quả triển khai

Họ và tên: Nguyễn Tuấn Anh

Lớp: E22CNPM-01

Mã sinh viên: B22DCDT019

Đề tài: [Healthcare Microservices]

Ngày 14 tháng 4 năm 2026

BÁO CÁO DỰ ÁN HEALTHCARE MICROSERVICES

Phân tích use-case, kiến trúc thiết kế, hiện thực mã nguồn và kết quả triển khai

Ngày 14 tháng 4 năm 2026

Tóm tắt nội dung

Tài liệu này mô tả báo cáo kỹ thuật cho dự án `healthcare-microservices`. Dự án mô phỏng hệ thống đặt lịch khám bệnh theo hướng microservice với ba miền nghiệp vụ cốt lõi: bệnh nhân, bác sĩ và lịch hẹn. Báo cáo đi từ bối cảnh bài toán, use-case, thiết kế kiến trúc, cấu trúc code theo hướng tách lớp đến kết quả triển khai trên Docker Compose.

1. Giới thiệu dự án và bối cảnh bài toán

Trong hệ thống y tế số, nghiệp vụ đặt lịch khám là một luồng cơ bản nhưng có nhiều ràng buộc dữ liệu: bệnh nhân phải tồn tại, bác sĩ phải tồn tại, thời gian hẹn phải hợp lệ và không được trùng lịch của bác sĩ. Nếu triển khai thiếu phân tách, logic nghiệp vụ dễ bị dàn trải trong controller hoặc phụ thuộc chặt giữa các module.

Dự án `healthcare-microservices` được xây dựng như một mô hình tham chiếu nhỏ gọn cho bài toán này. Hệ thống dùng nhiều service độc lập để mô phỏng cách một nền tảng healthcare có thể chia tách theo domain, đồng thời vẫn cho người dùng cuối một điểm truy cập thống nhất thông qua gateway.

1.1. Use-case chính

Các use-case chính của hệ thống gồm:

- Tạo hồ sơ bệnh nhân mới.
- Tạo hồ sơ bác sĩ mới kèm chuyên khoa.
- Tra cứu danh sách và chi tiết bệnh nhân, bác sĩ.
- Đặt lịch khám cho một bệnh nhân với một bác sĩ tại thời điểm xác định.

- Kiểm tra tính hợp lệ của lịch hẹn trước khi ghi nhận.
- Hiển thị toàn bộ lịch hẹn trên giao diện web.

2. Mục tiêu thiết kế

Hệ thống được thiết kế theo các mục tiêu sau:

- Tách riêng ba miền nghiệp vụ *patient*, *doctor*, *appointment*.
- Tổ chức lớp ứng dụng rõ ràng theo domain, application, infrastructure và interfaces.
- Tránh foreign key xuyên service; thay vào đó dùng HTTP call để xác nhận dữ liệu liên miền.
- Đảm bảo business rule của lịch hẹn được đặt tại **appointment-service**.
- Cung cấp giao diện web đơn giản để thao tác end-to-end.

3. Kiến trúc hệ thống

3.1. Các thành phần chính

Theo file `docker-compose.yaml`, hệ thống gồm năm thành phần:

Thành phần	Vai trò
gateway	Nginx gateway, vừa phục vụ frontend tĩnh vừa reverse proxy API đến các backend service.
patient-service	Django REST service quản lý dữ liệu bệnh nhân.
doctor-service	Django REST service quản lý dữ liệu bác sĩ và endpoint availability.
appointment-service	Django REST service xử lý nghiệp vụ đặt lịch và kiểm tra ràng buộc.
postgres	PostgreSQL dùng chung cho ba backend service.

3.2. Luồng xử lý

Luồng nghiệp vụ đặt lịch được tổ chức như sau:

1. Người dùng truy cập giao diện web tại `http://localhost:8080`.
2. Frontend gửi `POST /api/patients/` hoặc `POST /api/doctors/` để tạo dữ liệu nền.
3. Khi người dùng đặt lịch, frontend gửi `POST /api/appointments/`.
4. Gateway chuyển tiếp yêu cầu tới **appointment-service**.

5. `appointment-service` gọi sang `patient-service` để xác nhận bệnh nhân tồn tại.
6. `appointment-service` gọi sang `doctor-service` để xác nhận bác sĩ tồn tại và đang available.
7. Nếu hợp lệ, lịch hẹn được lưu vào bảng `appointments` trong PostgreSQL.

4. Thiết kế miền nghiệp vụ

4.1. Patient service

`patient-service` phụ trách quản lý thực thể bệnh nhân. Dữ liệu chính gồm:

- id dạng UUID.
- `full_name`.
- `phone`.
- `created_at`.

Các endpoint cốt lõi cho phép tạo, liệt kê và xem chi tiết bệnh nhân. Logic controller được giữ gọn, còn thao tác nghiệp vụ đi qua use-case và repository.

4.2. Doctor service

`doctor-service` quản lý thông tin bác sĩ với các trường:

- id dạng UUID.
- `full_name`.
- `specialty`.
- `created_at`.

Ngoài CRUD cơ bản, service này còn cung cấp endpoint `/doctors/<uuid>/availability/`. Trong phiên bản hiện tại, endpoint luôn trả về `available = true` nếu bác sĩ tồn tại và tham số thời gian hợp lệ. Đây là giả lập hợp lý cho một demo nhằm cho thấy cơ chế giao tiếp liên service.

4.3. Appointment service

`appointment-service` là trung tâm nghiệp vụ của dự án. Bảng `appointments` chứa:

- id dạng UUID.
- `patient_id`.

- `doctor_id`.
- `appointment_time`.
- `status`.
- `created_at`.

Mức cơ sở dữ liệu còn có ràng buộc `UniqueConstraint` trên cặp `doctor_id` và `appointment_time`. Điều này giúp đảm bảo không trùng lịch cùng bác sĩ ở cả tầng code lẫn tầng persistence.

5. Thiết kế mã nguồn theo lớp

5.1. Tổ chức thư mục

Điểm mạnh của dự án healthcare là cấu trúc thư mục khá sạch theo tư duy gần với Clean Architecture:

- **domain**: entity, repository abstraction, domain service.
- **application**: use-case điều phối nghiệp vụ.
- **infrastructure**: model Django, repository implementation, HTTP gateway.
- **interfaces**: serializer, URL và API view.

Nhờ vậy, business rule không bị nhúng trực tiếp vào controller.

5.2. Use-case đặt lịch

Business rule được tập trung trong `BookAppointmentUseCase`. Use-case này lần lượt kiểm tra:

1. Thời gian hẹn phải lớn hơn thời điểm hiện tại.
2. Bệnh nhân phải tồn tại.
3. Bác sĩ phải tồn tại.
4. Bác sĩ phải available tại thời điểm được chọn.
5. Bác sĩ chưa có lịch hẹn khác đúng thời điểm đó.

Listing 1: Logic cốt lõi của use-case đặt lịch

```
def execute(self, patient_id, doctor_id, appointment_time):
    if appointment_time <= timezone.now():
        raise ValueError("Appointment time must be in the future")
```

```

if not self.patient_gateway.exists(patient_id):
    raise ValueError("Patient does not exist")

if not self.doctor_gateway.exists(doctor_id):
    raise ValueError("Doctor does not exist")

if not self.doctor_gateway.is_available(doctor_id, appointment_time):
    raise ValueError("Doctor is not available")

if self.appointment_repo.exists_by_doctor_and_time(doctor_id,
appointment_time):
    raise ValueError("Doctor already has an appointment at this time")

```

Thiết kế này cho thấy việc kiểm soát nghiệp vụ được gom về một điểm duy nhất, thuận tiện cho bảo trì và mở rộng.

5.3. Gateway liên service

Trong `appointment-service`, hai lớp `PatientGateway` và `DoctorGateway` là cầu nối sang các service khác. Chúng gửi HTTP request dựa trên biến môi trường `PATIENT_SERVICE_URL` và `DOCTOR_SERVICE_URL`.

Listing 2: Gateway kiểm tra dữ liệu liên service

```

class PatientGateway:
    def exists(self, patient_id):
        response = requests.get(f"{PATIENT_SERVICE_URL}/patients/{patient_id}/",
        timeout=5)
        return response.status_code == 200

class DoctorGateway:
    def is_available(self, doctor_id, appointment_time):
        response = requests.get(
            f"{DOCTOR_SERVICE_URL}/doctors/{doctor_id}/availability/",
            params={"appointment_time": appointment_time.isoformat()},
            timeout=5,
        )
        return response.status_code == 200 and response.json().get("available",
False)

```

Đây là cách làm phù hợp với kiến trúc microservice, trong đó quan hệ dữ liệu được xác thực ở ranh giới service thay vì ORM join trực tiếp.

5.4. API layer gọn và rõ trách nhiệm

Lớp `interfaces/views.py` trong từng service giữ vai trò chuyển đổi HTTP request thành lệnh gọi use-case. Ví dụ, `AppointmentListCreateView` thực hiện validate serializer trước, sau đó khởi tạo use-case và trả mã lỗi 400 khi business rule không đạt.

Điểm này giúp controller mỏng, dễ đọc và tránh việc trộn lẫn logic framework với logic nghiệp vụ.

5.5. Gateway công khai bằng Nginx

Khác với dự án e-commerce dùng Django gateway, hệ healthcare dùng Nginx để reverse proxy các API path:

- `/api/patients/` tới `patient-service`
- `/api/doctors/` tới `doctor-service`
- `/api/appointments/` tới `appointment-service`

Listing 3: Ý tưởng routing tại Nginx gateway

```
location /api/appointments/ {
    proxy_pass http://appointment-service:8000/appointments/;
}
```

Giải pháp này tối giản nhưng hiệu quả cho môi trường demo, đồng thời tách rõ gateway hạ tầng khỏi logic nghiệp vụ backend.

5.6. Frontend tĩnh phục vụ demo toàn luồng

Frontend trong `gateway/frontend/app.js` dùng JavaScript thuần. Mặc dù đơn giản, giao diện vẫn đáp ứng đầy đủ chuỗi thao tác:

- Tạo bệnh nhân.
- Tạo bác sĩ.
- Đặt lịch khám.
- Đồng bộ danh sách lên giao diện.
- Ghi log thành công hoặc lỗi.

Đặc biệt, khi đặt lịch hệ thống tự chuyển `datetime-local` sang ISO UTC rồi gửi lên backend. Đây là chi tiết triển khai cần thiết để tránh sai lệch định dạng thời gian.

6. Triển khai và kết quả

6.1. Môi trường triển khai

Toàn bộ hệ thống được chạy bằng:

```
cd healthcare-microservices
docker compose up --build
```

Sau khi hoàn tất, gateway được public tại:

```
http://localhost:8080
```

Ba backend service và PostgreSQL chỉ chạy trong Docker network nội bộ, không mở trực tiếp ra host. Cách cấu hình này phù hợp với nguyên tắc đóng kín bề mặt tấn công và chỉ công khai một endpoint.

6.2. Kết quả đạt được

Dựa trên cấu trúc mã nguồn hiện tại, dự án đạt được các kết quả chính:

- Triển khai thành công luồng đặt lịch khám hoàn chỉnh theo kiến trúc microservice.
- Tách rõ ba miền nghiệp vụ bệnh nhân, bác sĩ và lịch hẹn.
- Áp dụng business rule tập trung tại `appointment-service`.
- Có cơ chế chống đặt trùng lịch ở cả tầng use-case lẫn cơ sở dữ liệu.
- Cung cấp giao diện web đơn giản nhưng đủ để demo nghiệp vụ end-to-end.
- Đóng gói toàn bộ hệ thống bằng Docker Compose, thuận lợi cho học tập và trình diễn.

6.3. Đánh giá kỹ thuật

Ưu điểm của dự án:

- Cấu trúc lớp rõ ràng, dễ mở rộng.
- Use-case tập trung, dễ kiểm soát business rule.
- Gateway công khai được giữ tối giản, backend service tách biệt tốt.
- Cách validate liên service phản ánh đúng tinh thần microservice.

Hạn chế hiện tại:

- Endpoint availability của bác sĩ hiện mới là mock logic đơn giản.
- Chưa có xác thực người dùng hoặc phân quyền.

- Chưa có hàng đợi sự kiện cho các thao tác hậu xử lý như thông báo xác nhận lịch khám.
- Chưa thấy hệ thống test tự động đủ dày để bao phủ các ràng buộc nghiệp vụ quan trọng.

7. Kết luận

`healthcare-microservices` là một mô hình microservice gọn nhưng có cấu trúc tốt cho bài toán đặt lịch khám bệnh. Giá trị nổi bật của dự án không chỉ nằm ở việc chia tách thành nhiều service, mà còn ở cách tổ chức mã nguồn theo lớp, cách kiểm tra ràng buộc nghiệp vụ trong use-case và cách triển khai toàn hệ thống thông qua một gateway duy nhất.

Nếu phát triển tiếp, dự án có thể nâng cấp theo các hướng như xác thực tập trung, cơ chế quản lý slot khám thực sự, event-driven notification, audit log và monitoring cho từng service.